

HYDROPONICS CONTROL SYSTEM

Redesign Concept & Engineering Specification

ESP32-S3 • Custom PCB • BLE Provisioning • FreeRTOS • MQTT • Local Fail-Safe Control

**Status: Proposed redesign based on the inherited prototype and handover documentation.
Proposed features are not presented as already implemented or validated.**

Prepared for project planning and documentation generation

September 2026

1. Executive Overview

The project is an overhaul of an existing hydroponics IoT prototype. The inherited system already demonstrated sensor acquisition, relay control, MQTT communication, dynamic runtime device instances, and integration with a Universal Gateway / EdgeX-style backend. However, the prototype remained breadboard-based, only one ESP node was validated end-to-end, several sensors were unfinished, and configuration/security/power architecture were still prototype-oriented.

The redesigned system will retain the useful dynamic device-management concept while moving toward a complete hydroponics monitoring and control platform centered on the ESP32-S3. The main engineering objective is a custom PCB, supported by a clean firmware architecture, BLE commissioning application, network dashboard, local fail-safe control, and better power design.

Core principle: essential hydroponic control must remain functional locally even when Wi-Fi, MQTT, or the backend is unavailable.

2. What We Inherit

Inherited element	Current status / interpretation
ESP32 firmware	Modular C++ firmware with one class per supported device type.
Dynamic device instances	Supported device classes can be instantiated, updated, or removed at runtime from configuration.
DeviceFactory	Maps a device profile/type to a precompiled C++ class.
MQTT	Used for telemetry and actuator commands.
Configuration server	Provides the ESP with its assigned device list.
Python tooling	Adds/removes profiles and devices and issues commands using JSON configuration.
Backend	Universal Gateway / EdgeX-style stack with MQTT and InfluxDB-related infrastructure.
Prototype hardware	ESP1 breadboard validated; ESP2-ESP4 intended but not physically proven.
Sensors	DHT22, light, TDS, pH, MQ-7, HC-SR04 and relay channels were part of the prototype.

3. Main Redesign Objectives

- Develop a custom PCB around the ESP32-S3.
- Complete the sensing and actuator hardware rather than leaving the system at breadboard stage.
- Add protected and deliberate 5 V / 3.3 V power distribution.
- Add battery / UPS backup for the controller and essential electronics.
- Use explicit FreeRTOS task separation instead of relying mainly on a monolithic application loop.
- Use BLE for commissioning, maintenance, diagnostics, and device management.
- Create a Python laptop application as a BLE GATT client.
- Move Wi-Fi/MQTT settings out of hard-coded source code and into persistent configuration.
- Provide a network-connected dashboard for real-time and historical monitoring.
- Retain local autonomous control and protection if network services fail.

4. Proposed System Architecture

Layer	Main responsibility
Sensors	Acquire pH, EC, water temperature, air temperature/humidity, reservoir level, and optional ambient-light data.
ESP32-S3	Sensor acquisition, filtering, calibration, local control, actuator management, BLE, Wi-Fi/MQTT, diagnostics.
Actuators	Pumps, valves, lights, fans or other loads via relays or MOSFET drivers.
BLE maintenance path	Commissioning, network setup, device creation, diagnostics and local live data.
Wi-Fi/MQTT path	Remote telemetry, commands, alarms, backend/database integration and dashboard.
Backend/dashboard	Visualization, history, supervision, trends, alarm display and safe manual controls.

Conceptual data/control flow:

Input / path	Controller	Output / path
Sensors	ESP32-S3 local controller	Actuators
Python BLE app	ESP32-S3 GATT server	Configuration / diagnostics
MQTT backend	ESP32-S3 network task	Telemetry / remote supervision

5. Sensor Strategy

Parameter	Preferred implementation	Purpose
pH	pH probe + suitable analog interface	Acidity/alkalinity monitoring and dosing logic.
EC	EC probe + EC interface	Primary indicator of overall dissolved-ion concentration.
TDS	Calculated/displayed from EC if desired	Optional user-facing ppm estimate; not the primary control variable.
Water temperature	Waterproof digital/analog probe	Water monitoring and EC temperature compensation.
Air temperature & humidity	Digital environmental sensor	Grow-environment monitoring.
Reservoir level	Appropriate level sensor	Low-water protection, refill logic and pump protection.
Ambient light	Optional	Lighting verification/monitoring if useful.

The inherited team used a TDS sensor but did not identify its exact model in the handover. The redesign should treat reuse of that module as an open decision. EC is preferred as the primary nutrient-solution concentration variable because the sensor fundamentally measures conductivity; TDS is typically a derived estimate.

6. Actuator Interface

Loads may use relays or MOSFET switching depending on voltage, current and switching requirements. DC pumps, valves, fans and LED loads are good candidates for N-channel MOSFET switching.

- The IRLZ44N may be useful for prototyping, but it is not the preferred final PCB choice unless 3.3 V gate performance is verified for the required current.
- Prefer a MOSFET with RDS(on) explicitly specified at approximately 2.5-3.3 V gate drive.
- Include a gate resistor and gate pull-down.
- Use a flyback diode across inductive DC loads where appropriate.
- Size copper, connectors and thermal dissipation from actual load current, not headline transistor current ratings.
- Define safe actuator states during boot, reset and communication failure.

7. Power & Backup Architecture

The final PCB should not use the development-board VIN pin as the general system power-distribution strategy. A dedicated power tree should be designed around the actual loads.

- Protected DC input with reverse-polarity protection.

- Regulated 5 V and 3.3 V rails as required.
- Local decoupling and bulk capacitance.
- Clear analog/digital/load grounding strategy.
- Separate high-current actuator branches where appropriate.
- Transient protection for inductive and externally wired loads.
- Battery / UPS support for ESP32, sensing, communications and essential control electronics.

Controller backup and actuator backup are separate engineering problems. A small battery can keep the controller alive; backing up high-power pumps or lights may require a much larger energy store.

8. Local Fail-Safe Control

Network communication is supervisory, not a prerequisite for basic safety. The ESP32-S3 must continue essential local control when Wi-Fi, MQTT or the backend is unavailable.

Function	Expected behavior during network loss
Sensor acquisition	Continue locally.
Safety rules	Continue locally.
Basic pump/light schedules	Continue when configured for local operation.
Low-water protection	Continue locally.
Maximum actuator run time	Continue locally.
Dashboard	May become unavailable.
Remote commands	Unavailable until reconnect.
Telemetry upload	Pause/buffer as practical and resume after reconnect.

- Reject or flag impossible sensor values.
- Provide maximum pump-on timeouts.
- Prevent dry running when reservoir level is unsafe.
- Define restart behavior after brownout or reset.
- Generate local alarms and publish them when communication is available.

9. FreeRTOS / ESP32-S3 Firmware Architecture

The ESP32 platform already uses FreeRTOS internally. The redesign should explicitly structure the application using tasks and inter-task communication rather than assuming that dual-core operation automatically improves performance.

Task / subsystem	Responsibility
------------------	----------------

Sensor acquisition	Timed readings, filtering, calibration and validity checking.
Local control	Setpoints, state machine, schedules and safety logic.
Actuator manager	Applies validated commands and safe-state policies.
Wi-Fi/MQTT	Connectivity, publish/subscribe, reconnect and backend communication.
BLE	GATT server, pairing, provisioning and local maintenance communication.
Device manager	Create/update/delete runtime device objects safely.
Diagnostics/logging	Health state, fault records and performance diagnostics.

Use queues, event groups, semaphores/mutexes and task notifications where they simplify concurrency. Core affinity should be applied only where it provides a measured benefit, for example to keep networking activity from disturbing time-sensitive acquisition/control.

10. BLE Commissioning & Python Desktop Application

The laptop application generates the graphical interface locally. The ESP32-S3 does not render the dashboard. The ESP exposes data and commands through BLE GATT.

Role	Device
BLE peripheral / GATT server	ESP32-S3
BLE central / GATT client	Python laptop application

- Discover nearby controllers.
- Securely pair with a selected controller.
- Display live sensor and status information.
- Configure Wi-Fi SSID/password.
- Configure MQTT/backend settings.
- Configure node identity.
- View the device list.
- Add, edit and delete supported runtime device instances.
- Display acknowledgements, validation errors and diagnostics.

Possible implementation stack: Bleak for BLE communication and PySide6/Qt for the desktop GUI.

11. GATT Design

GATT (Generic Attribute Profile) structures BLE data as services and characteristics. Characteristics may support Read, Write and/or Notify operations. For this project, GATT becomes the API contract between the ESP firmware and the Python maintenance application.

Service	Example characteristic	Operation
Telemetry	telemetry_data	Notify
Telemetry	system_state	Read / Notify
Network configuration	wifi_ssid	Read / Write
Network configuration	wifi_password	Write only
Network configuration	mqtt_server	Read / Write
Network configuration	node_name	Read / Write
Device management	device_list	Read
Device management	device_add	Write
Device management	device_update	Write
Device management	device_delete	Write
Device management	operation_result	Notify
Control / diagnostics	actuator_command	Write
Control / diagnostics	alarms	Notify

Avoid creating a very large number of characteristics. Telemetry may be packed into one structured binary or serialized message while configuration and commands remain clearly separated.

12. BLE Security

- BLE pairing and bonding.
- LE Secure Connections where supported.
- Encrypted GATT communication.
- Authenticated access for configuration and device-management writes.
- Provisioning mode activated by a physical button or limited commissioning window.
- Wi-Fi password should be writable but not normally readable back.
- Store persistent configuration securely in ESP32 non-volatile storage.

13. Dynamic Device Management

The useful part of the inherited system is retained: already-supported device classes can be instantiated at runtime. The redesign moves this workflow into the BLE desktop application instead of requiring manual JSON editing and separate scripts.

Step	Action
1	User selects a supported device type in the Python GUI.
2	App sends a GATT device-management request.
3	BLE task validates framing/security and queues the request.
4	Device Manager validates type, GPIO capability, conflicts and parameters.
5	DeviceFactory creates or updates the runtime object.
6	Persistent configuration is updated.
7	Result is returned to the application through a notification.

BLE callbacks should not directly create/delete objects that may currently be used by other tasks. Device-management requests should be serialized through an RTOS queue or equivalent synchronization mechanism.

- Use stable numeric/enumerated device-type identifiers internally.
- Allow creation/removal of instances without reflashing when the driver already exists.
- Adding a completely new sensor/actuator type still requires firmware support and recompilation.
- Validate GPIO conflicts, ADC capability, duplicate devices, required pin count and parameter ranges.

14. Network & Dashboard

BLE is intended mainly for commissioning and maintenance. Normal operation should use Wi-Fi/MQTT for remote supervision and data flow.

Dashboard function	Expected content
Live data	pH, EC, water temperature, air temperature/humidity, level and optional light.
Historical trends	Time-series graphs for the main measured parameters.
Actuator state	Pump, valve, light and other output states.
Health	Node connectivity, MQTT state, task/system status and

	fault indicators.
Alarms	Low water, invalid sensor, communication loss, timeout and other faults.
Calibration	Calibration date/state and service reminders.
Control	Safe manual commands and setpoint editing where appropriate.

The existing backend may be reused or replaced after evaluation. The important redesign requirement is a coherent operator interface rather than using MQTT Explorer as the primary dashboard.

15. Custom PCB Scope

- ESP32-S3 module/controller implementation.
- Protected input power and regulated rails.
- Battery/charging/backup interface.
- Sensor connectors and signal conditioning.
- pH and EC interfaces.
- Digital sensor buses.
- Relay and/or MOSFET output stages.
- Flyback/transient protection.
- RF keep-out and antenna layout for Wi-Fi/BLE.
- Grounding and analog-noise control.
- Current/thermal design.
- Programming/debug connector.
- Provisioning/reset buttons.
- Status LEDs.
- Test points.
- Serviceable field connectors.

16. Verification & Validation Plan

Area	Validation examples
Power	Rail accuracy, current margin, brownout behavior, battery switchover.
Sensors	Calibration, repeatability, range, disconnected-probe behavior.
EC/pH	Reference solutions, temperature compensation, drift.
Actuators	Load current, switching transients, fail-safe boot/reset state.

BLE	Pairing, encryption, provisioning, reconnect and unauthorized-write rejection.
Wi-Fi/MQTT	Reconnect behavior, broker loss, credential changes.
RTOS	Task timing, queue saturation, watchdog behavior, race-condition tests.
Fail-safe	Network loss, sensor fault, low-water state, brownout/restart.
PCB	Thermal checks, EMC/noise observations, connector robustness.

17. Open Engineering Decisions

- Exact ESP32-S3 module/package.
- Final EC probe/interface and whether the inherited TDS hardware is reusable.
- Final pH interface topology.
- Reservoir-level sensing technology.
- Relay versus MOSFET allocation per actuator.
- Nominal system input voltage and battery chemistry/capacity.
- Whether the existing EdgeX/Universal Gateway backend should be retained.
- Dashboard technology and data-retention strategy.
- Exact BLE packet format and UUID map.
- Configuration serialization format and NVS schema.

18. Documentation Deliverables to Generate

1. System Requirements Specification.
2. Existing-System Assessment.
3. Proposed System Architecture.
4. Hardware Block Diagram.
5. Power Architecture.
6. Sensor and Actuator Architecture.
7. PCB Design Requirements.
8. ESP32-S3 Firmware Architecture.
9. FreeRTOS Task Architecture and inter-task communication.
10. BLE/GATT Protocol Specification.
11. Python Desktop Application Architecture.
12. Dynamic Device-Management Specification.
13. Wi-Fi/MQTT Architecture.
14. Dashboard/Backend Requirements.
15. Local Fail-Safe and Fault-Handling Requirements.
16. Security Requirements.
17. Calibration Strategy for pH and EC.

18. Testing and Validation Plan.

19. Open Technical Decisions and Risks.

19. Source Basis & Status Notes

This concept document is based on the inherited June 2026 Hydroponics ESP32 handover report and the older Universal Gateway documentation. Those sources support the description of the existing prototype, including ESP1 validation, dynamic device instances, MQTT/gateway architecture, hard-coded network settings, and the original sensor set. The redesign sections above are proposed engineering requirements and should not be read as already implemented or validated.

The old documentation also contains legacy infrastructure credentials. Those credentials should be treated as exposed and replaced if the associated systems remain active.